

Ultimate Computer Software Security Audit Report

Risks, evidence, and hardening recommendations

Report date: May 10, 2026

Scope: Local source-code security review of frontend, backend, Prisma schema, routes, auth, uploads, SEO/public endpoints, dependencies, and deployment configuration.

Important limitation: This is a source-code security review, not a full penetration test. Findings should be validated again in the deployed production environment.

Executive Summary

- No obvious deliberate public backdoor was found, such as an unauthenticated admin route, hardcoded login bypass, database dump endpoint, or active raw SQL injection path.
- The highest practical risks are default admin credentials, arbitrary admin-managed ad HTML, broad iframe/embed permissions, and production environment misconfiguration.
- The application already has several strong controls: HTTP-only auth cookies, CSRF protection, hashed refresh sessions, hashed reset tokens, Prisma query safety, and backend admin role checks.
- The recommended launch posture is to fix the high-risk items first, then address metadata leaks, session hardening, and abuse-rate limits before public production use.

Risk Register

The table below summarizes all identified risks and the primary recommended action for each.

Severity	Finding	Primary action
High	Default admin credentials remain in seed/docs	Pre-launch fix.
High	Admin-controlled ad HTML is rendered publicly	Pre-launch fix.
High	Article body iframes allow high-risk third-party embed hosts	Pre-launch fix.
High	Production protections depend heavily on NODE_ENV=production	Pre-launch fix.
Medium	Refresh token rotation has a concurrency race	Next hardening pass.
Medium	OG image endpoint can leak unpublished article metadata	Next hardening pass.
Medium	Public series endpoint can expose draft article metadata	Next hardening pass.
Medium	Poll endpoints can expose unpublished article polls	Next hardening pass.
Medium	Article view endpoint can be abused to inflate reads	Next hardening pass.
Medium	2FA recovery codes are shorter than recommended	Next hardening pass.
Medium	Disabling 2FA does not revoke active refresh sessions	Next hardening pass.
Medium	Dormant data processing functions could be dangerous if exposed later	Next hardening pass.
Low	Frontend dependency advisory for Quill	Routine cleanup.
Low	Frontend public robots.txt allows all routes	Routine cleanup.
Low	Accidental empty .crdownload file in public folder	Routine cleanup.

Severity	Finding	Primary action
Low	Upload pre-filter trusts client MIME type	Routine cleanup.

Detailed Findings

High Risk

Default admin credentials remain in seed/docs

Severity: High

Evidence: backend/prisma/seed.ts:113-126; README.md:118-121

Risk: If the seeded admin account or password is still valid in production, an attacker can log in as an administrator.

Recommendation: Change the admin password immediately, remove default credentials from public docs, disable seed-admin creation in production, and require 2FA for admins.

Admin-controlled ad HTML is rendered publicly

Severity: High

Evidence: frontend/src/components/AdBanner.tsx:59;
backend/src/controllers/configController.ts:35-47

Risk: A compromised admin account or unsafe ad snippet can become stored XSS or a malicious third-party script vector across public pages.

Recommendation: Replace arbitrary HTML ads with structured ad settings, sanitize tightly, or render ads inside sandboxed iframes with a strict allowlist.

Article body iframes allow high-risk third-party embed hosts

Severity: High

Evidence: backend/src/utils/sanitize.ts:177-189; frontend/src/pages/public/ArticlePage.tsx:85-96

Risk: Code execution platforms and broad iframe allowlists increase phishing, tracking, and content-execution risk inside article pages.

Recommendation: Allow only YouTube by default. If code demos are required, sandbox every non-YouTube iframe with minimal permissions and remove broad host regexes.

Production protections depend heavily on NODE_ENV=production

Severity: High

Evidence: backend/src/app.ts:93-101; backend/src/middleware/rateLimiters.ts:15-25;
backend/src/config/env.ts:36-69

Risk: If production is deployed with NODE_ENV=development, rate limits and CAPTCHA defaults are weaker, increasing brute-force and abuse risk.

Recommendation: Add startup checks for production deployments and fail closed unless NODE_ENV, HTTPS cookies, Turnstile, strong secrets, and proxy settings are correct.

Medium Risk

Refresh token rotation has a concurrency race

Severity: Medium

Evidence: backend/src/controllers/authController.ts:302-339;
backend/src/services/refreshSessionService.ts:68-99

Risk: Two simultaneous refresh requests can both validate before revocation, reducing replay-detection strength.

Recommendation: Rotate tokens with an atomic conditional update inside one transaction and require exactly one affected row before creating the next session.

OG image endpoint can leak unpublished article metadata

Severity: Medium

Evidence: backend/src/controllers/ogController.ts:145-159

Risk: A known draft/submitted slug can reveal title, author, and category through /og/article/:slug.

Recommendation: Require status=PUBLISHED for public OG image generation unless caller is authenticated as owner/admin.

Public series endpoint can expose draft article metadata

Severity: Medium

Evidence: backend/src/controllers/articleController.ts:822-843

Risk: Series responses include member article status, title, slug, excerpt, and image even if articles are not published.

Recommendation: Filter public series members to published articles. Show unpublished members only to owner/admin.

Poll endpoints can expose unpublished article polls

Severity: Medium

Evidence: backend/src/controllers/pollController.ts:89-124 and 220-255

Risk: Polls linked to unpublished content may be retrievable by ID or article ID.

Recommendation: When a poll has articleId, verify linked article visibility before returning poll data.

Article view endpoint can be abused to inflate reads

Severity: Medium

Evidence: backend/src/controllers/articleController.ts:497-575

Risk: Anonymous clients can repeatedly POST view events, inflating stats and causing database write load.

Recommendation: Add IP/user rate limits, anonymous dedupe window, capped timeSpentSeconds, and optional bot/captcha protection for suspicious bursts.

2FA recovery codes are shorter than recommended

Severity: Medium

Evidence: backend/src/utils/totp.ts:101-110

Risk: Current recovery codes are 32-bit each, which is weaker than typical backup-code entropy.

Recommendation: Generate stronger one-time recovery codes, for example 10 to 16 random bytes per code, and keep only hashes in the database.

Disabling 2FA does not revoke active refresh sessions

Severity: Medium

Evidence: backend/src/controllers/authController.ts:481-534

Risk: Existing sessions remain valid after a major security setting changes.

Recommendation: Revoke all refresh sessions after enabling/disabling 2FA, password reset, role changes, bans, and other high-risk account actions.

Dormant data processing functions could be dangerous if exposed later

Severity: Medium

Evidence: backend/src/services/dataProcessing.ts:13-257

Risk: The service can export or deactivate accounts based only on email if a future route calls it without strong verification.

Recommendation: Remove unused dangerous functions or require authenticated/verified identity plus admin approval and audit logging.

Low Risk

Frontend dependency advisory for Quill

Severity: Low

Evidence: npm audit --omit=dev in frontend reports quill XSS advisory through react-quill-new

Risk: Rich text editor dependency has a known low-severity advisory.

Recommendation: Track upstream fix, update when compatible, and keep server-side article sanitization as the primary defense.

Frontend public robots.txt allows all routes

Severity: Low

Evidence: frontend/public/robots.txt

Risk: If the frontend static robots file overrides the backend robots endpoint, admin/private routes are not disallowed for crawlers.

Recommendation: Align frontend robots.txt with backend robots rules and disallow /admin, /dashboard, /login, /register, /reset-password, /api/admin, and /api/me.

Accidental empty .crdownload file in public folder

Severity: Low

Evidence: frontend/public/Unconfirmed 716710.crdownload

Risk: The file is empty, but public folders should not contain accidental browser download artifacts.

Recommendation: Delete accidental public files and add ignore rules/checks to keep them out of builds.

Upload pre-filter trusts client MIME type

Severity: Low

Evidence: backend/src/middleware/upload.ts:24-35;
backend/src/controllers/uploadController.ts:23-33

Risk: Sharp rejects invalid images later, but MIME type alone is client-controlled and should not be the first trust boundary.

Recommendation: Validate magic bytes and Sharp metadata before processing; add pixel-count limits to reduce decompression-bomb risk.

Strong Controls Already Present

- Authentication uses HTTP-only cookies plus CSRF protection.
- Access-token verification re-checks the user and role from the database.
- Refresh tokens are stored as hashes in database-backed sessions.
- Password reset tokens are hashed, expiring, and single-use.
- Prisma is used for database access; no practical raw SQL injection pattern was found in active code.
- Admin routes are protected by backend authentication and role middleware.
- Public contact/data requests are queued instead of automatically exporting or deleting accounts.
- Article body HTML is sanitized server-side and again client-side before rendering.

Priority Remediation Plan

1. Change and rotate the admin password; remove default credentials from docs and seed output.
2. Require 2FA for admin and moderator accounts before production launch.
3. Replace raw ad HTML with safe structured configuration or sandboxed iframe rendering.
4. Restrict article embeds to YouTube or sandbox all non-YouTube embeds with minimal permissions.
5. Fail startup when production security environment variables are missing or unsafe.
6. Fix public metadata leaks in OG images, series, polls, pinned profile articles, and notification hydration.
7. Make refresh-token rotation atomic and revoke sessions after security changes.
8. Add abuse limits to views, comments, reactions, follows, polls, and article-request endpoints.
9. Clean public/static folders and align robots.txt with backend rules.

Production Launch Security Checklist

- NODE_ENV=production
- HTTPS enforced at the proxy/CDN
- COOKIE_SECURE=true
- COOKIE_SAME_SITE=lax or strict unless cross-site auth is required
- JWT_SECRET and JWT_REFRESH_SECRET generated with high entropy and never committed
- TWO_FACTOR_ENCRYPTION_KEY set separately from JWT secrets
- CAPTCHA_REQUIRED=true and CAPTCHA_SECRET configured
- TRUST_PROXY correctly set behind the production reverse proxy
- Admin and moderator accounts protected by 2FA
- Database backups encrypted and access restricted to production operators only
- Centralized logs enabled for auth failures, admin actions, password resets, bans, and report status changes

Audit Commands Used

Representative local checks used during the review:

- `npm audit --omit=dev`
- `rg "dangerouslySetInnerHTML|eval|localStorage|document.cookie|child_process|$queryRaw"`
- `rg "TODO|FIXME|backdoor|bypass|Admin@12345"`
- `find . -name ".env*" -print`